

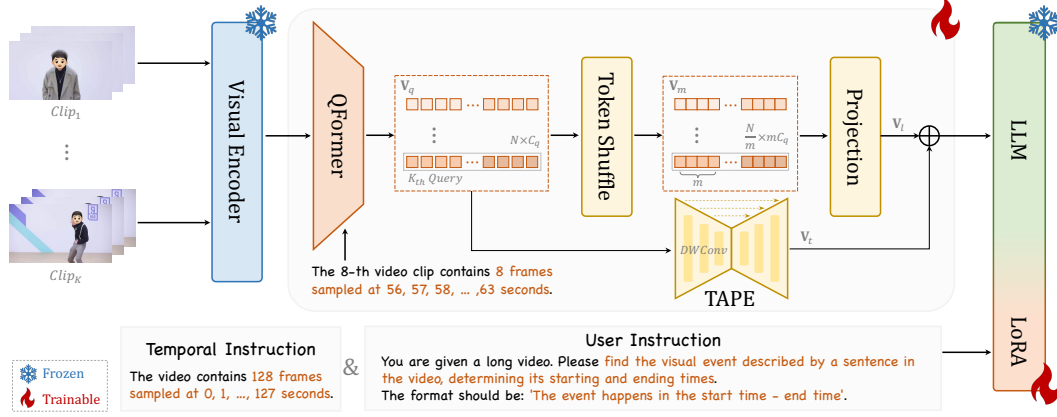


Figure 2: **Overall Architecture of VideoChat-T.** First, long videos are segmented into clips, which are then transformed into feature embeddings by video encoder and time-aware Qformer. Next, all visual tokens undergo Token Shuffle to compress overly long tokens, and generate adaptive positional encodings through TAPE. Finally, the long video tokens are concatenated with the user query, serving as the input of LLM, thereby generating appropriate responses.

3.1.1 BACKBONE DESIGN

Video clip encoding. For the given long video, we perform uniform sampling (Wang et al., 2019) to obtain $K \times T$ frames. We divide these frames into K video segments in chronological order, and sample T frames from each segment. Next, we use the video encoder and its visual-linguistic connector (Q-Former here) to encode each segment into N tokens. After the aforementioned processing, the entire video is encoded into a sequence of visual tokens, denoted by $\mathbf{V}_q \in \mathbb{R}^{L \times C_q}$, where C_q is the dimension of output token by the Q-Former and $L = K \times N$ is the total number of tokens for the entire video.

Large Language Model. According to previous research, images and visual cues are projected into the same feature space of the LLM. The LLM acts as an interaction interface in the MLLMs, being used to process multimodal inputs, parse user instructions, and generate appropriate responses. To afford the processing of long video sequence, we need to design an efficient compression module between the visual encoder and LLMs.

3.1.2 VL-CONNECTOR: TOKEN SHUFFLE

The increased number of sampled frames in long videos leads to a larger number of encoded visual tokens, causing a significant rise in the computational complexity and memory consumption of LLMs. Therefore, it is crucial to keep the number of visual tokens within an acceptable range. Some works have proposed various token compression schemes, such as clustering (Jin et al., 2024) and pooling (Huang et al., 2024b). However, clustering methods often struggle to maintain the temporal consistency, and pooling methods usually result in a certain loss of overall performance.

To address this, we propose a simple token shuffling compression scheme that ensures the temporal consistency of video tokens before and after compression while avoiding excessive performance loss. Previous methods often used a projector to achieve dimensional conversion. However, projecting visual encoding vectors from low to high dimensions does not increase information density. Therefore, we propose to rearrange multiple visual tokens along the channel dimension. Specifically, for the long video $\mathbf{V}_q = [v_q^1, v_q^2, \dots, v_q^L] \in \mathbb{R}^{L \times C_q}$, we concatenate m adjacent tokens along the channel dimension to obtain the reshaped visual feature $\mathbf{V}_m = [v_m^1, v_m^2, \dots, v_m^{\frac{L}{m}}] \in \mathbb{R}^{\frac{L}{m} \times m C_q}$, where each merged token v_m^i is represented as:

$$v_m^i = \text{Concat}(v_q^{(i-1)*m+1}, v_q^{(i-1)*m+2}, \dots, v_q^{i*m}) \quad \forall i = 1, 2, \dots, \frac{L}{m}.$$

Next, a linear projection layer is applied to the merged visual feature $\mathbf{V}_m$, generating the visual token sequences $\mathbf{V}_l \in \mathbb{R}^{\frac{L}{m} \times C_l}$ as input into the LLM, where C_l represents the token channel di-

mension of the LLM. This scheme effectively reuses the projector of base model by replicating the original linear layer parameters m times along the channel dimension, achieving an initialization equivalent to mean pooling with a window length of m . This design avoids introducing additional randomly initialized parameters that might disturb the original model, thus preserving its original capabilities. Additionally, compared to directly using pooling, this method offers higher flexibility for fine-tuning to achieve better results (see ablation study, Table 4).

3.1.3 TEMPORAL ADAPTIVE POSITION ENCODING

To bind temporal positional information to visual tokens, we propose an adapter called **Temporal Adaptive Position Encoding (TAPE)**. Inspired by CPVT (Chu et al., 2021), our TAPE uses zero padding at both ends of the convolution as anchors, and gradually transmits relative positional encoding information. Without the need to add any special time tokens, TAPE can automatically perceive the relative temporal positions of the token sequence and generate temporal embeddings.

Specifically, the long video token sequence $\mathbf{V}_q$ is first compressed in the channel dimension by a linear layer and further compressed in sequence length by a pooling layer. Next, we use a U-Net-like structure composed of one-dimensional depthwise separable convolutions to progressively down-sample the sequence, obtaining three one-dimensional temporal feature sequences with different resolutions. Subsequently, a convolution with a sufficiently long window is applied to the shortest temporal feature sequence, using zero padding at both ends as anchors to encode the relative temporal position of each token in the sequence (Chu et al., 2021). Then, we progressively upsample and restore the temporal feature sequences from short to long, using residual connections to retain temporal features at different scales. Finally, the temporal feature sequences are restored to the same length as $\mathbf{V}_l$ and aligned in the channel dimension by a linear layer, thereby obtaining the temporal features $\mathbf{V}_t$ output by the TAPE. For detailed implementation of TAPE, please refer to Appendix A.

Our proposed TAPE offers a plug-and-play module, which could be easily integrated into the network structure via residual connections, adding temporal position information to video tokens without disrupting the distribution of other trainable parameters. With appropriate training strategies, TAPE effectively preserves the model’s generalization capabilities and enhances its temporal sensitivity (see ablation study, Table 3), which is important for temporal grounding task.

3.2 TIMEPRO: TEMPORAL GROUNDED INSTRUCTION DATA

Traditional temporal grounding datasets only contain monotonous ground truth, i.e., the start and end times of the target period. This data format performs well in training the classic expert models, but is difficult to unleash the potential of LLMs. Although several temporal grounding-centric datasets have been released for MLLM fine-tuning (Ren et al., 2024; Huang et al., 2024b), they still have deficiencies in data quantity, data quality, and task diversity. Thus, it is necessary to build a more comprehensive temporal dataset designed for the tuning of MLLMs.

Based on the criteria of diversity, length, and difficulty, we collect and clean several existing high-quality grounding-centric datasets (Ren et al., 2024; Huang et al., 2024a;b), and create two new datasets, resulting in the TimePro. Compared to previous temporal grounding-centric datasets, TimePro offers a larger volume of data, a broader distribution, and a higher task diversity, facilitating the learning of more generalizable temporal representations for MLLMs.

As shown in Figure 3(a), TimePro contains 9 task types from 15 datasets that are highly relevant to temporal grounding, containing approximately 349K high-quality temporal grounding annotations. The 9 tasks are specified as follows. **Temporal Video Grounding** involves identifying the start and end times of video content based on a natural language query (Anne Hendricks et al., 2017; Oncescu et al., 2021; Zala et al., 2023). **Dense Video Captioning** requires detecting events within a video and providing corresponding timestamps and descriptions (Krishna et al., 2017; Huang et al., 2020; Zhou et al., 2018). **Video Summarization** focuses on determining key frames or clips in the form of timestamps rather than semantic summaries (Song et al., 2015; Gygli et al., 2014). **Step Localization** aims to segment and describe important steps in a long video (Tang et al., 2019; Zala et al., 2023). **Transcribed Speech Generation** predicts speech content and its timestamps from visual signals (Zellers et al., 2022). **Reasoning Temporal Localization** combines timestamps with explanatory answers (Huang et al., 2024b). **Multi-format Temporal Grounding** includes single-turn and multi-turn dialogues with diverse question types (Huang et al., 2024a). **Highlight**

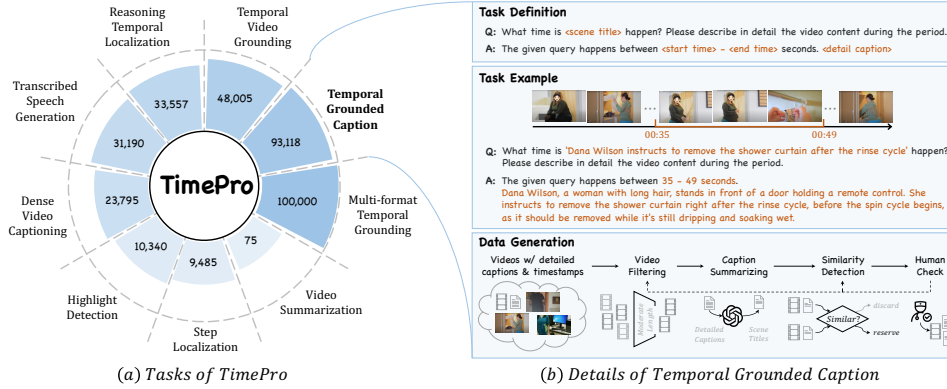


Figure 3: (a) The proposed temporal centric instruction-tuning dataset, **TimePro**. This dataset contains approximately 349K high-quality and strongly temporally correlated data. (b) The proposed Temporal Grounded Caption fine-tuning data paradigm. It effectively reducing the occurrence of hallucinations. We employ a 4-stage processing pipeline to ensure the quality of the generated data.

Detection identifies the most significant moments in a video based on a query (Lei et al., 2021a). **Temporal Grounded Caption** uses a brief scene title to output both the time period and fine-grained description for the scene. More detailed information about TimePro is available in the appendix B. It should be noted that Temporal Grounded Caption is our newly-designed task that can help our model to establish fine-grained correspondence between visual segment and linguistic description.

3.3 TEMPORAL GROUNDED CAPTION TASK

Some studies have shown that MLLMs often exhibit severe hallucinations when dealing with fine-grained perception tasks (Ji et al., 2023; Huang et al., 2023; Golkar et al., 2023). Since our VideoChat-T directly regresses the timestamps corresponding to the text queries using MLLMs, it is more susceptible to hallucinations compared to methods that use external expert models as decoders (Wu et al., 2024). By forcing the video MLLMs to predict the event occurrence time and simultaneously describe the visual content evidence, we attempt to anchor these queries to the relevant time segments within the video, rather than generating hallucinations originating from LLM itself. Based on this analysis, we design the Temporal Grounded Caption task.

The top of Figure 3(b) illustrates the definition of Temporal Grounded Caption. We use a brief scene title of the video segment as the query, requiring the model to simultaneously respond with the precise start and end times of the video segment and provide a detailed description of that segment. While the content in the scene title may leak into the detailed caption response, most of the missing detailed information must be correctly described by attending the corresponding segment. Moreover, temporal grounding and detailed captioning can serve as regularization task for each other, preventing caption model from hallucinations from unrelated visual or linguistic contexts and helping grounding model to regress the timestamp more accurately.

The process for collecting our Temporal Grounded Caption data is described at the bottom of Figure 3(b). In the first stage, we use a detailed caption dataset with timestamps as our data source. We remove data with target grounding time intervals that are too short or too long and ensure that the scenes in the video are as diverse as possible. In the second stage, we use a LLM to summarize scene titles. To prevent excessive semantics of video segments from being leaked from the query to the MLLM, we try to retain the minimal subset of key features that are sufficient to distinguish the video segments. In the third stage, to avoid overly similar or identical content appearing at different temporal intervals in the video, we perform similarity filtering on the data annotations. Based on the scene titles and video features, we calculate the similarity between different segments of the same video and remove data with excessively high similarity. In the fourth stage, we randomly sample the generated data and manually assess its quality. Based on human feedback, we refine the threshold parameters for data filtering used in the first three stages to yield the final Temporal Grounded Caption dataset. This new dataset plays an important role in our grounded tuning.